Entropy-Based Compression Using Huffman Coding

Analysis and Implementation in Go

Carson Beissel

California State University San Marcos

Marcus Chen

4/03/2026

Abstract

This paper describes a brief history of compression algorithms and the design and implementation of a Huffman coding algorithm in Golang as an entropy coding approach to lossless data compression. Entropy coding is a lossless data compression scheme that reduces data size by assigning more frequent byte values to shorter bit representations, producing a more compact bitstream. This program was tested on a variety of files and data formats, including but not limited to random bytes, English-language text, MP3, and more, within the paper. The algorithm will be evaluated in terms of average symbol size, execution time, and the number of bytes written before vs. after. Results of the algorithm can be exceptional or, in some edge cases, worse than the initial data size based on the input data size, format and structure.

Entropy-Based Compression Using Huffman Coding Analysis and Implementation in Go

There are over 402.74 million terabytes or 402 quintillion bytes created every single day. Approximately 82% of that data consists of video content that has to be streamed or delivered to a device via the internet (Howarth, 2025). Given the magnitude of data, it's quite clear why the concept of compression becomes critical, especially today, with the growing requirements and strain put on infrastructure from AI. With critical data, the concepts of lossless and lossy data are important for maintaining the exponential growth in demand. Lossless compression guarantees that a file before compression will be absolutely identical to the file after decompression. Entropy coding is a form of lossless compression that has many different implementations. At a high level, this looks like translating data into bitstreams with a smaller symbol size representation than that of the source. As a result, this is seen as an improvement to data storage as the files become smaller. David A. Huffman was an accomplished computer scientist with important contributions to information theory, coding, signal design, and communications (IEEE Computer Society, n.d.), most notably the Huffman coding algorithm. Conceptually, Huffman coding works by creating a binary tree with leaves closest to the root pertaining to symbols with the highest frequency. The tree is traversed such that each left branch will add a '0' and each right branch will add a '1' to the symbol's bitstream encoding. Golang was the primary implementation language chosen because of its ease of cross-platform compilation, performance, simplicity, and personal familiarity and favorite. This paper presents the design, implementation, and evaluation of the Huffman coding compression efficiency across various forms of data.

Literature Review

The framework for data compression originated with Claude Shannon and his 1948 paper "A Mathematical Theory of Communication", where Shannon established the statistical limits of

theoretical data compression. He defined entropy as the probability of the distribution of the symbols, which became a foundation for modern Information Theory and entropy coding methods (Shannon, 1948).

Huffman Coding

After the foundation was created by Shannon, Huffman built directly on top of Shannon's work with his paper "A Method for the Construction of Minimum-Redundancy Codes" (Huffman, 1952). Huffman analyzes the frequency of bytes and constructs relative frequency tables, and describes the Huffman coding algorithm. The algorithm works by constructing a priority queue based binary tree, which assigns a unique variable length code to each specific value of a byte or symbol. This is one of the main strengths of the Huffman tree because it allows for the data to be decoded unambiguously, offering methods for lossless data transmission.

Compression Today

Although created in 1952, Huffman's coding is still widely prevalent today in its many implementations in modern data compression industry standards, such as deflate. Deflate is a compression algorithm that is the modern basis for popular formats such as ZIP and GZIP files. It is built on LZ77 compression with entropy based Huffman coding (Deutsch, 1996). Despite the age of Huffman coding, the wide use of these file formats and more is evidence of the impact it still has today.

Implementation Language

The choice of Go (Golang) as the implementation language for this project is motivated by its design characteristics and my personal preference for the language. Created by Google in 2009 (Griesemer et al., 2009), Go is a statically typed compiled language that creates native machine code, eliminating the need for virtual machines or interpreters. It's as dynamic as

Python and as fast as C/C++. Although not necessary in this implementation, it is still important to mention one of the main selling points that Go is built around concurrency and multithreaded programming with goroutines. It has a simple syntax and is exceptionally well rounded for implementing Huffman's algorithm.

Method

Program Design & Implementation

Encoding

The program was designed in a way that it could be dropped into any codebase and executed within a couple of function calls. There are only two necessary functions to call to encode a file. The first is the initialization function for the Huffman encoder table. This function takes in a file path as an argument and constructs a key-value map, mapping raw byte values regardless of type from the file to the frequency with which they appear as an integer.

Figure 1

Key-Value table

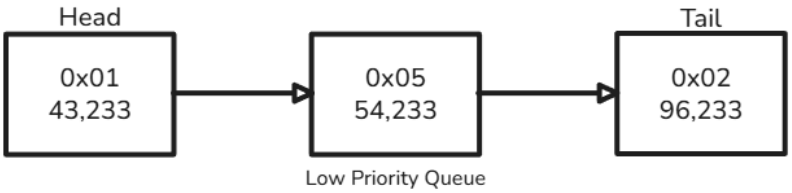
Key	Value
0x4e	23,341
0x0b	10,129
0x0b	3,493

After the creation of the map, the program creates a minimum priority queue implemented with a linked list. A linked list was chosen as the implementation data structure, as the maximum size of unique bytes is limited to 256. This means that even as the file grows extremely large, the performance of the algorithms will not be affected by memory locality significantly enough to matter. At most, the function is traversing 256 memory locations, which may or may not be already cached by the CPU. The linked list also offers a simple underlying structure for

implementing a queue as well as a stack, rather than using a circular static array or other primitive structure that would require more complex functions for indexing.

Figure 2

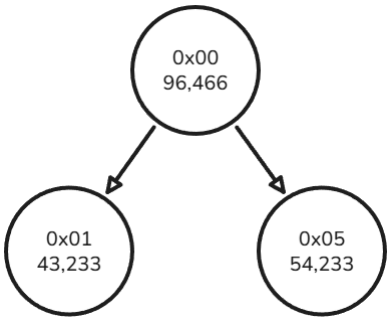
Minimum Priority Queue Implementation as a Linked List



A new node is created from each byte value and frequency from the map and enqueued. Once the queue is created, it is time to execute the Huffman coding algorithm. First, the algorithm will dequeue the first two nodes from the queue. As it's a minimum priority queue, it will always be sorted from least to greatest or, in this case, least frequent to most frequent. The first two nodes are guaranteed to be those of the lowest frequent bytes. After dequeueing, it will then create a new internal node with the two dequeued nodes as the children to this new internal node and frequency as the combined frequency of the children.

Figure 3

Internal Node Created From the Lowest Two Frequency Nodes

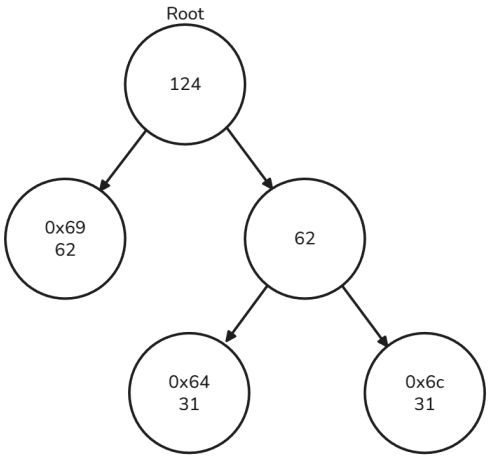


After the internal node is created, it is enqueued back into the queue. This process repeats until there is only one node left. At this point, the function finally dequeues the last node and sets this

as the root of our Huffman tree, the total or root frequency being the total size of the file initially in bytes.

Figure 4

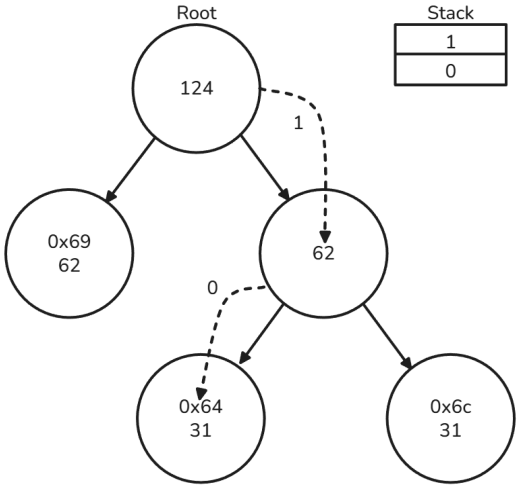
Huffman Tree Example



Next is creating our encoding table, which is a map that takes a key as a byte value and returns a binary path to that byte value in the Huffman tree. This is done by a modified recursive inorder traversal from the root of the Huffman tree, which, in addition to passing a node pointer, also passes pointers to a stack and a map of node pointer key to binary values. The stack is used to keep track of the path taken toward the leaf nodes. A ‘0’ is appended to the stack when a left child is taken, and a ‘1’ when a right child is taken.

Figure 5

Huffman Tree Encoding Path



If both children are nil, then the current node is a leaf and must have a byte value attached to it. It then sets this leaf node's pointer within the map to the binary path. After this, a simple iterator function is used to create a new map or our final encoding table by iterating through the map node pointer map to create a new map that directly converts raw byte values into binary paths.

Figure 6

Encoding Table Mapping Byte Values to Binary Paths

Encoding Table

Byte Value	Binary Path
0x69	0
0x64	10
0x6c	11

After the encoding table has been initialized, it then goes on to encode the given input file from the initial function call. The input file is opened along with a newly created encoded file to be written into while the input file is being read. The encoded file is prefixed with some important information regarding the format of the data within it. The first byte of the file is

reserved for the bit padding count, along with the second byte reserved for the number of key value pairs in the encoding table. Then the file is written in with the entire coding table so the decoder can use it later. The format used is byteValue,encodedByteValue|byteValue, encodedByteValue|... and so forth. After the file prefixing, the input file is read byte by byte, converting bytes into their binary path, forming a bitstream that's written into the output file. For the final bits written, if they do not form a complete byte, the amount of padding is calculated and written into the first reserve byte position of the file. As the issue of writing a partial amount of bits into a byte will result in the last byte value being incorrectly written.

Decoding

The decoding function is much shorter than the encoder. First, it starts with opening the file from the input argument of the function call. It first reads in the two reserved bytes for the padding count and size of the encoding table, respectively. After this, it iterates through the encoding table within the file, recreating the table in memory. Once the decoding table has been read in, it is then used to start decoding the file. It starts by calculating the total amount of bits within the file, subtracting the padding bits at the end from the reserve byte. Then, iterating through every byte and then every bit, mapping the bitstream to a key in the table, and writing the translated byte into the output decoding file.

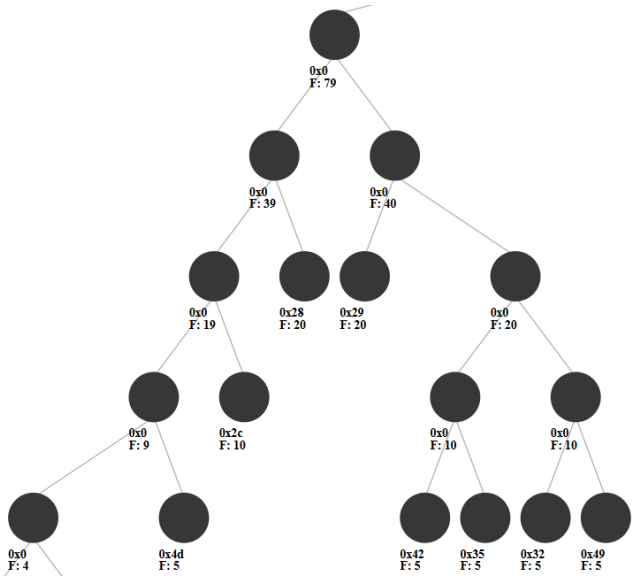
Viewer

The viewer is a flag that can be passed into the program to create an HTML document with an interactive graphical representation of the Huffman tree that was created to encode the input file. It includes a pan and zoom to navigate the tree as well as the following metrics: the original file size (bits), encoded size (bits), encoder table size (bytes), total bytes written, theoretical improvement percentage, actual improvement percentage, and average symbol size.

This function is standalone and only requires a root node and an optional metric struct from the encoder. At a high level, it executes an inorder traversal, writing SVG elements to the document, keeping track of node x, y positions, and offsetting so there are no visual collisions.

Figure 7

SVG HTML Viewer Screenshot



Test Data

The program was evaluated on a variety of file types and sizes used to represent a large range of byte distributions. English language text was chosen from famous works because of the skewed nature of some letters appearing vastly more often than others, for example, ‘e’ vs. ‘z’. Randomly generated byte files of various sizes were also included in the test suite, generated from a small program written in Python, seeded with the built-in random library. These files act as a sort of control, as the entropy of the files is maximized and should show the worst-case scenario possible for the algorithm. MP3 and PNG files were selected because they have already been compressed with other lossy or lossless methods. They will act as another form of control, as they should resist further compression due to the nature of entropy. BMP formatted files were

selected as they are raw image data files that are not compressed. These files are expected to have lower entropy. This would then offer the greatest percentile gain for compression due to repeating colors and sequences in images. Lastly, the test suite included various program source code from open-source projects in 5 different languages, as another form of low-entropy file format, since programming languages tend to skew character distributions. Think of the amount of brackets, semicolons, and parentheses in languages like C and C++, or the number of times you have to write in an if err != nil statement in Go.

Table 1

Test BMP Files Used for Compression

File Type	Description	Size
BMP	400x300x24	351.68KB
BMP	400x300x32	468.87KB
BMP	512x128x16	128.07KB
BMP	512x128x24	192.05KB
BMP	512x128x32	256.07KB
BMP	Rat Icon	27.05KB
BMP	Alien	75.74KB
BMP	America Big Moose	679.39KB
BMP	Black & White Photo	30682.34KB
BMP	4x1	0.13KB
BMP	4x2	0.14KB
BMP	A4X1	0.13KB
BMP	A4X2	0.15KB
BMP	TV Color Test 1	17.05KB

BMP	Boy	385.05KB
BMP	Car	40.41KB
BMP	Compositing	62.62KB
BMP	Eye	801.24KB
BMP	Flag	45.1KB
BMP	Fond RVB	244.19KB
BMP	Fundo Vetor	732.47KB
BMP	HQX Original 1	129.99KB
BMP	HQX Original 2	214.18KB
BMP	Text Scheme	50.68KB
BMP	Stained Glass	190.48KB
BMP	Landscape	769.05KB
BMP	Landscape 2	769.05KB
BMP	Landscape 3	150.12KB
BMP	Woman	768.05KB
BMP	Woman Resized	48.05KB
BMP	Woman 1	1024.07KB
BMP	Woman 2	1024.07KB
BMP	Woman 3	1024.07KB
BMP	Woman 3 Resized	768.05KB
BMP	Woman 512x512	257.05KB
BMP	Woman Unedited	768.05KB
BMP	Marbles	4176.34KB
BMP	Marie	797.22KB
BMP	Mountain	301.05KB

BMP	News Guy	297.05KB
BMP	Night Sky	81.05KB
BMP	Mountain Lion	296.71KB
BMP	Children	3600.06KB
BMP	Doctor Office	257.05KB
BMP	Parrots	288.05KB
BMP	Icebergs	273.18KB
BMP	Graphics Raytracing Test	1410.3KB
BMP	Lingatures	24.05KB
BMP	Context Menu	900.05KB
BMP	Sails	385.05KB
BMP	Ocean Sunset	271.69KB
BMP	Spheres	281.3KB
BMP	Terrain	65.05KB
BMP	TV Color Test 2	16.06KB
BMP	TV Color Test 3	16.06KB
BMP	TV Color Test 4	117.24KB
BMP	Movie AD	213.48KB
BMP	Pattern Test	41.38KB
BMP	Line Art	35.68KB
BMP	256 Bit Color Array	48.05KB
BMP	Venus	45KB
BMP	Windows Pipes	33.8KB
BMP	CAD	33.8KB
BMP	W3C Home Icon	10.18KB

BMP	Capital Building Wallpaper	20709.36KB
BMP	Black &White Weather Lady	297.05KB
BMP	Xing Sample	115.37KB

Table 2

Test English Text Files Used for Compression

TXT	The Adventures of Huckleberry Finn	570.82KB
TXT	Hamlet, Shakespeare	176.4KB
TXT	Moby Dick, Melville	1205.99KB
TXT	Romeo & Juliet, Shakespeare	139.26KB
TXT	Comp. Nums. Application To Entscheidungs Problem, A.M. Turing	69.65KB

Table 3

Test MP3 Files Used for Compression

MP3	First Movement From Symphony No. 5 In C Minor, Op. 67, Beethoven	12327.61KB
MP3	Minuet In G, Beethoven	4227.05KB
MP3	Für Elise, Beethoven	4843.6KB
MP3	Turkish March From Ruins Of Athens, Beethoven	2814.73KB
MP3	Final Movement From Symphony No. 9 In D Minor, Op. 125, Beethoven	33366.86KB

Table 4

Test PNG Files Used for Compression

PNG	Moon Lit Forest Skyline	726.18KB
PNG	Galaxy	9700.14KB
PNG	Mountain Valley View	1527.53KB
PNG	Train Tracks	1838KB
PNG	Aurora Borealis	1512.82KB

Table 5

Test Program Source Code Files Used for Compression

GO	“buffer.go” Golang Compiler Source Code	17.01KB
CC	“charset.cc” C Compiler Source Code	108.93KB
HS	“general.hs” Haskell Compiler Source Code	27.12KB
Zig	“link.zig” Zig Compiler Source Code	92.86KB
RS	“mod.rs” Rust Compiler Source Code	29.8KB

Evaluation Metrics

The metrics evaluated are as follows: number of bits, number of encoded bits, table size, percent improvement theoretical, percent improvement actual, average symbol size, and execution time. The formula simply calculates the number of bits:

$$B_r = s \cdot 8$$

Where B_r is the number of bits from the raw input file, and s is the number of bytes in the file. Next is the number of encoded bits; encoded bits are the raw bits written from the bitstream when encoding. This importantly does not include the table or any reserve bytes and is calculated as follows:

$$B_e = \sum_{i=0}^{n-1} l_i \cdot f_i$$

B_e is the number of encoded bits, l_i is the length of the Huffman code in bits, and f_i is the frequency of the byte i in the input data. Next is the number of total bytes written, which is

calculated by calling the stat function on the output file after completion which we will call B_t.

Next is the average symbol size, which is the number of encoded bits divided by the total number of symbols.

$$\bar{S} = \frac{\sum_{i=0}^{n-1} l_i \cdot f_i}{\sum_{i=0}^{n-1} f_i}$$

Where $\bar{S}$ is the average length or symbol size, the numerator is the total number of encoded bits, the denominator is the total number of symbols or bytes from the input file, and L is the length of the average symbol size in bits. Next is the table size, which is calculated by the following:

$$S_t = 2 + \sum_{i=0}^{n-1} 3 + l_i$$

Where S_t is the total table size or length and l_i the length of the path. The number 3 is chosen because it represents the table format of the raw byte value before encoding the pipe and comma in between the length of the path. The number 2 represents the two reserve bytes required for decoding. Next is the theoretical percent improvement.

$$I_{theo} = 1 - \frac{B_e}{B_r} = 1 - \frac{\sum_{i=0}^{n-1} l_i \cdot f_i}{s \cdot 8}$$

Where I_{theo} is the percentage improvement in decimal form, B_e is the number of encoded bits, B_r is the number of total bits from the input file, l_i is the length of the Huffman code in bits, f_i is the frequency of the byte i in the input data, and s is the number of bytes in the file. This measures the ratio between raw bits and encoded bits and a theoretical percent improvement without

including the size of the decoder table and reserve bytes. Next is the percent improvement actual, which measures the total encoded file size over the file size before encoding.

$$I_{real} = 1 - \frac{B_t}{B_r} = 1 - \frac{B_t}{s \cdot 8}$$

Where I_{real} is the realized actual percentage improvement in decimal form, B_t being the number of total written encoded bytes, including the table and reserve, B_r is the number of bits from the raw input file. Next is Shannon's entropy function.

$$H = -\sum_{i=0}^{n-1} p(x) \cdot log_2(p(x))$$

Where p(x) is the probability of each symbol or byte value appearing. The result H is in bits per symbol, which represents the theoretical minimum average bits required to encode each byte.

Testing Environment

All tests were conducted on Windows 10 Home with a Ryzen 3600X 3.8GHz processor and 32GB of DDR4 3200MHz memory. All Golang binaries were built with the default flags, i.e., "go build ." and tests were conducted while the PC was under minor load (25-35% CPU Usage) from other processes.

Testing Procedure

The testing procedures were conducted on every file from the provided tables 20 times each. The execution time was then averaged from these tests, which was measured solely from the start of creating the Huffman encoder until the output file was completely written. Tests were conducted by another program in Golang that would iterate through every file in the testing directory, going through every file one by one, iteratively, and recording data into a results.csv file.

Results

Table 6

Images BMP

Description	Original Size (KB)	Compressed Size (KB)	% Actual Improvement	Avg Symbol Size	Avg Runtime (ms)
400x300x24	351.68	308.17	0.115466	7.010241	30.45
400x300x32	468.87	343.34	0.261431	5.858107	38.4
512x128x16	128.07	65.54	0.486328	4.093769	7.95
512x128x24	192.05	60.02	0.68665	2.500315	11.55
512x128x32	256.07	70.52	0.723871	2.203307	15.1
Rat Icon	27.05	10.04	0.535449	2.969786	3.05
Alien	75.74	28.37	0.62109	2.996506	5.15
America Big Moose	679.39	559.36	0.17239	6.586617	61.55
Black & White Photo	30682.34	28514.56	0.070558	7.43478	2659.2
4x1	0.13	0.03	-0.216418	1.671642	0.6
4x2	0.14	0.04	-1.321918	2.39726	0.65
A4X1	0.13	0.03	-0.217391	1.710145	1.05
A4X2	0.15	0.05	-1.37013	2.519481	1.1
TV Color Test 1	17.05	6.45	0.493185	3.025999	2.6
Boy	385.05	327.76	0.140837	6.809761	37
Car	40.41	30.14	0.179764	5.965758	5.3
Compositing	62.62	61.83	-0.032284	7.899981	7.75
Eye	801.24	796.63	0.002279	7.953998	80.15
Flag	45.1	8.32	0.813694	1.475705	3.25
Fond RVB	244.19	140.11	0.414019	4.590292	25.7

Fundo Vetor	732.47	409.89	0.436026	4.476803	53.1
HQX Original 1	129.99	75.64	0.400669	4.655285	10.2
HQX Original 2	214.18	128.14	0.387426	4.786291	17.1
Text Scheme	50.68	14.97	0.647019	2.36248	4.65
Stained Glass	190.48	176.73	0.057046	7.422473	20.6
Landscape	769.05	679.14	0.113009	7.064673	75.5
Landscape 2	769.05	696.99	0.089905	7.250375	98.95
Landscape 3	150.12	105.96	0.289081	5.646639	15.9
Woman	768.05	720.19	0.058402	7.50143	84.95
Woman Resized	48.05	47.82	-0.052859	7.960696	6.5
Woman 1	1024.07	824.85	0.191489	6.443745	92.45
Woman 2	1024.07	786.42	0.229542	6.143503	94.45
Woman 3	1024.07	823.73	0.192586	6.434938	91.05
Woman 3 Resized	768.05	720.19	0.058402	7.50143	76.7
Woman 512x512	257.05	240.34	0.052708	7.47988	28.25
Woman Unedited	768.05	713.64	0.067463	7.433269	70.4
Marbles	4176.34	3654.69	0.124185	7.000751	369.45
Marie	797.22	762.59	0.039788	7.652496	72.2
Mountain	301.05	183.1	0.37957	4.865479	27.35
News Guy	297.05	271.34	0.076513	7.307612	30.25
Night Sky	81.05	67.9	0.123762	6.702053	8.85
Mountain Lion	296.71	287.69	0.02089	7.756627	29.6
Children	3600.06	3439.83	0.043708	7.643941	340.45
Doctor Office	257.05	146.27	0.415926	4.552123	23.55
Parrots	288.05	269.89	0.052803	7.49544	28.8

Icebergs	273.18	232.06	0.140408	6.79587	27
Raytracing Test	1410.3	787.94	0.440996	4.469619	94.55
Lingatures	24.05	9.1	0.572919	3.027324	2.35
Context Menu	900.05	374.15	0.580584	3.325557	57.4
Sails	385.05	299.57	0.21382	6.224082	38.55
Ocean Sunset	271.69	232.76	0.132452	6.853713	29.6
Spheres	281.3	269.54	0.031577	7.665368	30.15
Terrain	65.05	47.22	0.225305	5.807092	7.15
TV Color Test 2	16.06	14.77	-0.100851	7.355562	3.1
TV Color Test 3	16.06	15.13	-0.119271	7.534407	2.9
TV Color Test 4	117.24	111.42	0.02558	7.602977	12.75
Movie AD	213.48	55.17	0.738786	2.067455	14.3
Pattern Test	41.38	33.01	0.128604	6.380976	5.2
Line Art	35.68	8.04	0.77098	1.802896	2.85
256 Bit Color Array	48.05	18.52	0.612385	3.082937	3.85
Venus	45	19.11	0.484635	3.397001	4.8
Windows Pipes	33.8	28.99	0.053158	6.861732	4.4
CAD	33.8	25.3	0.159878	5.988762	4.1
W3C Home Icon	10.18	3.14	0.594032	2.466897	1.4
Capital Building	20709.36	19562.08	0.055262	7.556809	1939
Black &White	297.05	273.34	0.071178	7.361336	27.2
Xing Sample	115.37	107.4	0.043696	7.447568	12

Table 7

English Text

Description	Original Size (KB)	Compressed Size (KB)	% Actual Improvement	Avg Symbol Size	Avg Runtime (ms)
The Adventures of Huckleberry Finn	570.82	328.73	0.424117	4.593561	44.95
Hamlet, Shakespeare	176.4	108.34	0.385829	4.882394	15.65
Moby Dick, Melville	1205.99	685.68	0.431439	4.538202	98.4
Romeo & Juliet, Shakespeare	139.26	84.87	0.390554	4.835658	14.9
Comp. Nums. Application To Entscheidungs Problem, A.M. Turing	69.65	44.32	0.363675	4.93535	8.1

Table 8

Music MP3

Description	Original Size (KB)	Compressed Size (KB)	% Actual Improvement	Avg Symbol Size	Avg Runtime (ms)
First Movement From Symphony No. 5 In C Minor, Op. 67, Beethoven	12327.61	12324.58	0.000245	7.996248	1311.85
Minuet In G, Beethoven	4227.05	4227.02	0.000008	7.994724	425.4
Für Elise, Beethoven	4843.6	4844.53	-0.00019	7.99697	508.15
Turkish March From Ruins Of Athens, Beethoven	2814.73	2816.28	-0.00055	7.99657	290.95

Final Movement From Symphony No. 9 In D Minor, Op. 125, Beethoven	33366.86	33345.18	0.00065	7.994141	3348.9

Table 9

PNG files

Description	Original Size (KB)	Compressed Size (KB)	% Actual Improvement	Avg Symbol Size	Avg Runtime (ms)
Moon Lit Forest Skyline	726.18	728.54	-0.003237	7.995537	74.75
Galaxy	9700.14	9702.89	-0.000284	8	919
Mountain Valley View	1527.53	1529.34	-0.00119	7.99508	154.15
Train Tracks	1838	1840.07	-0.001126	7.997009	188.2
Aurora Borealis	1512.82	1513.21	-0.000254	7.987421	150.8

Table 10

Source Code Text

Description	Original Size (KB)	Compressed Size (KB)	% Actual Improvement	Avg Symbol Size	Avg Runtime (ms)
buffer.go Golang Compiler Source Code	17.01	11.47	0.325545	4.957979	3
charset.cc C Compiler Source Code	108.93	71.24	0.345989	5.149367	10.15
general.hs Haskell Compiler Source Code	27.12	17.15	0.367669	4.729829	3.75
link.zig Zig Compiler Source Code	92.86	53.33	0.425729	4.501435	9.25
mod.rs Rust Compiler Source Code	29.8	17.65	0.407721	4.454581	3.65

Table 11

Random Byte Values

Description	Original Size (KB)	Compressed Size (KB)	% Actual Improvement	Avg Symbol Size	Avg Runtime (ms)
1,000 Random Bytes	0.99	1.77	-0.793855	6.601586	1.5
10,000 Random Bytes	9.87	9.19	0.068757	6.678967	2.4
50,000 Random Bytes	49.36	42.28	0.143569	6.697114	6.4
250,000 Random Bytes	246.62	207.67	0.157937	6.705635	25.8
1,000,000 Random Bytes	986.46	828.3	0.160329	6.70964	94.15
1,000,000,000 Random Bytes	986328.06	827623.83	0.160904	6.712759	93088

Figure 8

Average Actual Compression Improvement by File Type

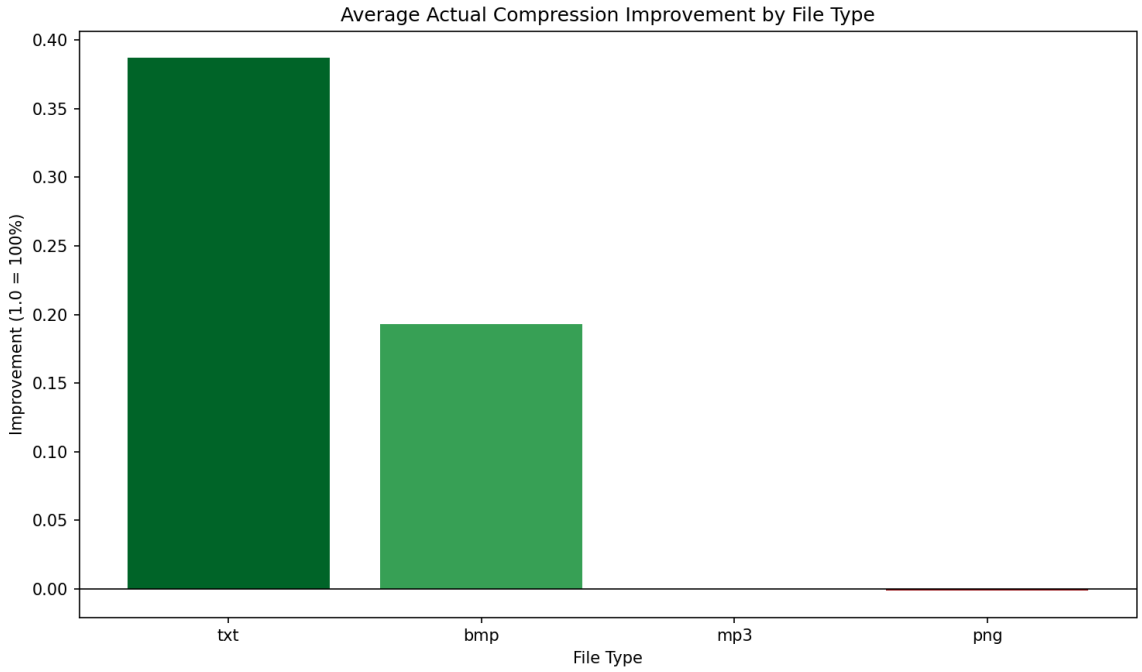
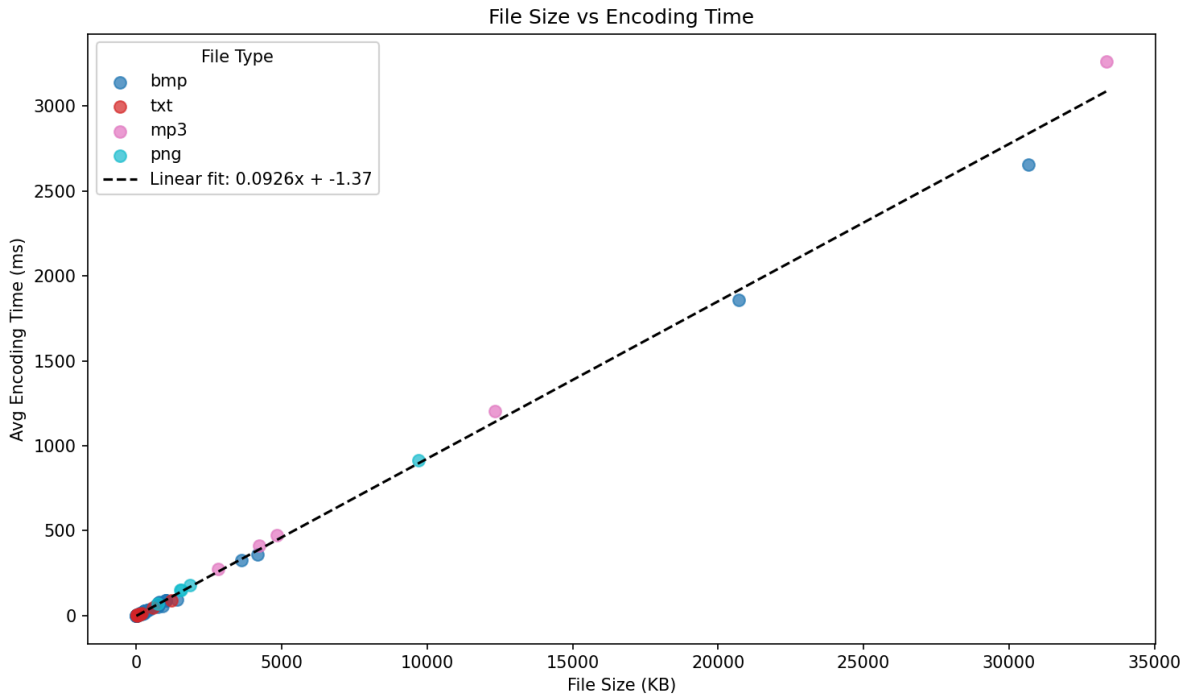


Figure 9

File Size vs. Encoding Time



A total of 93 files were tested across 6 distinct file types. 67 BMP images, 5 TXT files, 5 source code files, 5 MP3s, 5 PNGs, and 6 random byte value files. Overall, the algorithm achieved great compression ratios where files were uncompressed initially. Performance of the algorithm's compression metrics varied significantly between file types, depending on the structure, with 16 of the 93 files producing a negative gain. The encoder and decoder functions both run at O(n) time complexity, as demonstrated in Figure 9.

BMP Image Files

BMP files demonstrated a wide range of compression results for any type tested. The best performing was flag_b24.bmp, which was able to reduce the size of the file by 81.37% with an average symbol size of 1.48 bits, being able to represent a byte. Another strong performer was Line Art with 77.1% improvement and Flag with 73.88%. The average improvement amongst all

BMP files was approximately 20%. However, there were several extremely small BMP files that produced ‘compressed’ output larger than the initial input. The worst was bmp_a4x2.bmp, which increased the size by 137.01%. These files were extremely small in size, which led to the Huffman encoding table being prefixed into the file, ending up larger than the data it was trying to represent.

Text Files

Text files were divided into three main categories: English language literature, programming source code, and random byte sequences. As expected, English literature had the strongest and most consistent compression results from all the text categories. Moby Dick by Herman Melville had the highest compression ratio at 43.14% with an average symbol size of 4.54 bits and Shannon entropy of 4.5 bits. The Adventures of Huckleberry Finn by Mark Twain was similar with 42.41% and with an average symbol size of 4.59 bits with a Shannon entropy of 4.56 bits. Hamlet had 38.58% with an average symbol size of 4.88 bits and Shannon entropy of 4.85 bits. Romeo and Juliet had a 39.06% improvement with 4.84 average bit size per symbol with a Shannon entropy of 4.8 bits. Alan Turing’s Computational Numbers with Application to Entscheidungs Problem achieved a 36.37% improvement with an average symbol size of 4.94 bits and Shannon entropy of 4.898 bits. The average symbol size across all literary texts ranged from 4.54 bits to 4.94 bits, a small range well below the initial 8 bits per byte input and extremely close to the theoretical limit of data compression.

Programming source code files were similar, with a range of 32.55% for a Go source file to 42.57% with the Zig source file. Average symbol sizes ranged from 4.45 to 5.15 bits, slightly more significant than literary text.

Random number text files performed the worst, as expected. The smallest file, 1000.txt, increased in size by 79.39%, again due to the Huffman table prefix in the encoded file being larger than the encoded data itself. Larger random byte files were able to achieve small gains ranging from 6.88% and 16.09% with average symbol sizes ranging around 6.7. This is expected since random bytes have extremely high entropy.

MP3 files showed on average 0% gains. Results ranged from 0.07% to a 0.06% negative gain and an increase in file size. Average symbol sizes stayed extremely close to 8 bits across every single file. This is an expected outcome since MP3 files are already compressed using similar lossy methods that include the Huffman algorithm within them.

PNG files were consistently the worst performers in creating outputs slightly larger than the original input file. With all five files showing a negative improvement with a range of -0.03% to -0.32%. Average symbol sizes, as expected, were extremely close to 8 bits. PNG is an already compressed file format and uses Deflate, which incorporates Huffman coding under the hood. This attempt at double compressing then shows there is no added benefit, and the added encoding table prefix within the newly compressed files resulted in consistently larger outputs.

Overall, of the files tested, 16 produced compressed outputs that were larger than the initial input. These were due to 3 main reasons. Either they were very small files, where the table size was larger than the actual input, already compressed formats like MP3 and PNG, where the table once again would increase the file size as no further compression could be achieved, or random

data where entropy was extremely high with no skewing for the Huffman coding to take advantage of.

Discussion

The measured results from the data set demonstrate the real effectiveness of Huffman coding and clearly show that the efficiency of the compression is directly related to the frequency distribution of the input data. This is consistent with the initial findings in the literature review and foundational information that was established by Shannon's first paper on the topic. Files with highly skewed byte distributions offer the greatest gains in compression, while files with high entropy or uniform distribution show minimal gains or, in some cases, with tiny files, an increase in size.

English language literary texts achieved the greatest and most consistent improvements across the board. This is expected because of the structure of the English language, and words having a skew toward common letters, such as 'e', 'o', and 't', will appear far more likely than letters like 'z' or 'x'. Even this paper itself, if subjected to a byte distribution frequency table and analyzed, will further prove this. This skew is what essentially allows the Huffman tree to assign short bit paths to the most common characters like 'e', which, as a result, will significantly reduce the final average symbol size. The range of bits per symbol across the various files stayed very close together, further proving letter skewing as well as getting impressively close to Shannon's theoretical limit for compression.

Another form of English text tested was source code files. There was a range of improvement from 32.55% to 42.57%. The slightly lower range and average are caused by the wider range of characters required in a programming language. For example, brackets, parentheses, operators, tildes, and more. This requires the Huffman tree to have more leaves and,

in turn, more paths to account for these extra characters, reducing the overall effectiveness of the compression. However, the point still stands that programming languages do in fact have characters that appear absolutely everywhere, in C++, parentheses, semicolons, newlines, and spaces are everywhere.

Random number text files were included to show limitations and further prove the correlation between the byte frequency and overall performance. Because random bytes, as the file size increases, are going to become more uniformly distributed, the algorithm will hit a plateau in compression gains. This is exactly what was observed with small files, we ran into the recurring issue of the table taking up more space than the compression gains offer. Once the file size grows to megabytes and even a gigabyte, the algorithm plateaus at around a 16% compression gain.

BMP image files produced a wide range of results, from an 81.37% to a 137.01% increase. This is due entirely to the initial file size and distribution of pixel data within the file. BMP files are uncompressed pixel data and are directly related to the visual content of the image. If the entire image were red, then that would be optimal for compression, but an image where each pixel is a different color would result in being the worst case. Because of this, small files where the 137.01% increase was seen were due to the Huffman encoding table prefixed to the encoded file, which took up more room than the encoding could provide gains, as well as the combination of the distribution of pixel data being high in entropy.

MP3 and PNG files showed virtually no compression benefit, with the average symbol sizes hovering around 8 bits. This result confirms and justifies the choice in use, as using these file formats as a form of controlled variable, verifying the implementation as well as theory. Applying a Huffman coding to already compressed data is extremely ineffective. MP3 and PNG

already use entropy coding internally, with PNG using Deflate, which already incorporates its own Huffman coding, as well as MP3 using Huffman coding in its final stage of encoding. Thus, the remaining byte distributions of these files are already as evenly distributed as possible, leaving virtually no room for further compression with this algorithm.

Limitations

This study has several limitations that are worth making a note of. Primarily, this paper focuses on Huffman coding alone. In many compression algorithms today, like ZIP, GZIP, and Deflate, multiple compression methods are combined to achieve greater gain. For example, ZIP uses a dictionary method named LZ77, eliminating repeated sequences, then applying Huffman coding. Testing Huffman coding in isolation is able to measure its standalone abilities, however it does not reflect real world compression performance where multiple methods are combined.

Another limitation of the study is the dataset used. The dataset, although chosen to be diverse to show both edge cases of the algorithm and a middle ground, is not robust enough. A larger dataset with hundreds of file formats over thousands of different files with hundreds of tests each would have provided much more valuable data and insight.

Another limitation is in the implementation; the implementation of preseeding every compressed file with the translation can be improved. The format is wasteful, especially in the use of its commas and pipes. These are wasted bytes that could be eliminated with a more complicated algorithm for storing and retrieving the data required to decompress the rest of the file. This would lead to further improvements and allow for smaller files to have actual measurable compression gains rather than losses due to the table.

Future Works

This paper shows the implementation, design, and evaluation of the Huffman encoding algorithm written in Golang as an entropy coding based approach to lossless data compression in isolation. The program was tested across 93 files, consisting of BMP images, English language in TXT files, source code files, MP3 audio, PNG images, and random byte files. The results concluded that the algorithm's effectiveness is directly correlated to the byte frequency distribution of the input file's data, consistent with prior research. English language text files in the form of famous literary works achieved the greatest and most consistent improvements, with up to a 43.14% compression gain and average symbol size of just 4.54 bits. This approaches the Shannon entropy limit of approximately 4.5 bits per symbol. The already compressed file formats, such as PNG and MP3, showed virtually no gain or loss, confirming and solidifying prior theories. Huffman coding cannot be applied twice for further meaningful gains. In conclusion, this paper takes a general overview of the Huffman coding and shows how it is an effective method for entropy coding when applied to datasets with a skewed frequency distribution. Go proved to be a capable implementation language that offers clear, easy to follow syntax and code. Future work to extend this project could incorporate an implementation for the LZ77 dictionary compression and combine it with this implementation of Huffman Coding, and benchmark it against actual compression algorithms.

References

Deutsch, P. (1996). *DEFLATE compressed data format specification version 1.3* (RFC 1951). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc1951

Griesemer, R., Pike, R., Thompson, K., Taylor, I., Cox, R., Kim, J., & Langley, A. (2009, November 10). *Hey! Ho! Let's Go!* Google Open Source Blog. https://opensource.googleblog.com/2009/11/hey-ho-lets-go.html

Howarth, J. (2025, April 24). *Amount of data created daily (2025)*. Exploding Topics. https://explodingtopics.com/blog/data-generated-per-day

Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE, 40*(9), 1098–1101.

IEEE Computer Society. (n.d.). *David A. Huffman*. https://www.computer.org/profiles/david-huffman

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal, 27*(3), 379–423. https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf